



Informatica per Scienze e Tecnologie Ambientali

Note del Corso

Ultimo aggiornamento: 8 marzo 2026

Antonio Mario Caruso

Università del Salento

✉ antonio.caruso@unisalento.it 🏠 <https://antoniocaruso.github.io/it/>

🌐 <https://github.com/antoniocaruso/>

Week 1

Suggerimenti su queste Note

Queste note contengono una piccola guida agli argomenti trattati durante le lezioni ed il materiale su Internet da trovare e studiare.

❗ Le note **Non Sono** esaustive, e non possono essere usate come **unica** fonte.

L'esame si svolge in modalità Orale, la parte relativa alle vostre conoscenze si programmerà, si svolgerà al calcolatore, durante l'orale.

Si **raccomanda** vivamente ogni studente, di spendere tempo a sufficienza al calcolatore, nello studio dei problemi che saranno presentati a lezione.

Piramide di Astrazione

Applicazioni (software come QGIS, Excel, etc)

Algoritmi e Linguaggi di Programmazione (Python¹), Linguaggi Formali vs Naturali.

Sistema Operativi: (Interfacce GUI, Terminale, Servizi per il Programmatore)

Reti di Calcolatori: Internet, Architettura Client-Server, Cloud, Internet of Things, Environmental Sensor Networks.

Hardware: Rappresentazione dell'Informazione e Architettura di un Calcolatore

¹<https://www.python.org>

Programmazione in Python

Faremo subito la conoscenza di un linguaggio di Programmazione, la rete offre centinaia di risorse per il linguaggio vedi:

https://do1.dr-chuck.com/pythonlearn/IT_it/pythonlearn.pdf

Si consiglia vivamente di accedere comunque a materiale in inglese per abituarsi e per la qualità più elevata dello stesso.

Se avete un portatile, *portatelo!!*, almeno nelle prime lezioni, in modo da poter configurare correttamente l'ambiente di programmazione.

Studiare il Capitolo del libro disponibile presso:
<https://shorturl.at/nbvIN>

Visti a Lezione: Sezione 2.1, Approfondimenti: 2.2, 2.7.

Definizione di *Bit*

Approfondimento 2.7.

Dimensione Minima codice Binario

Theorem

Data una sequenza di $k \geq 1$ bit, sia C il numero massimo di configurazioni diverse rappresentabili, allora $C = 2^k$.

Dimensione Minima codice Binario

Theorem

Data una sequenza di $k \geq 1$ bit, sia C il numero massimo di configurazioni diverse rappresentabili, allora $C = 2^k$.

Dimostrazione.

Per induzione su k . Caso base: $k = 1$, allora ovviamente abbiamo solo due diverse configurazioni associate a $b = 0$ o 1 .

Consideriamo le sequenze di k bit: sia b_k il valore del bit più significativo, possiamo partizionare queste sequenze in due gruppi, quelle con $b_k = 0$, e quelle con $b_k = 1$. Per induzione la dimensione di ogni gruppo è pari a 2^{k-1} e poichè sono due gruppi, il numero totale sarà $2^{k-1} * 2 = 2^k$. ■

Dimensione Minima codice Binario

Theorem

Data una sequenza di $k \geq 1$ bit, sia C il numero massimo di configurazioni diverse rappresentabili, allora $C = 2^k$.

Dimostrazione.

Per induzione su k . Caso base: $k = 1$, allora ovviamente abbiamo solo due diverse configurazioni associate a $b = 0$ o 1 .

Consideriamo le sequenze di k bit: sia b_k il valore del bit più significativo, possiamo partizionare queste sequenze in due gruppi, quelle con $b_k = 0$, e quelle con $b_k = 1$. Per induzione la dimensione di ogni gruppo è pari a 2^{k-1} e poichè sono due gruppi, il numero totale sarà $2^{k-1} * 2 = 2^k$. ■

Lemma

Per rappresentare N oggetti diversi, la dimensione minima in bit di un codice binario è pari a $k = \lceil \log_2 N \rceil$.

Week 2

Argomenti (Lunedì 10)

- Numeri Interi (rappresentazione modulo e segno), complemento a due (solo cenni).
- Numeri Floating-Point, standard IEEE-754, singola e doppia precisione, errori di troncamento, errori delle operazioni (esempio in Python `0.1+0.1+0.1==0.3` $\rightarrow$ `False`).
- Basi diverse: numeri ottali e esadecimali.
- Grandezze Analogiche: digitalizzazione, quantizzazione, campionamento. Frequenza di Campionamento, Teorema di Nyquist²

²https://it.wikipedia.org/wiki/Teorema_del_campionamento_di_Nyquist-Shannon

Argomenti (Mercoledì 12)

Calcolo Automatico, Storia Antica: Abaco, Pascalina, Telaio di Jacquard, Macchina Analitica e Differenziale di Babbage.

Personaggi: Pascal, Babbage, Ada Lovelace, Alan Turing, John von Neumann, Moore (legge di).

Macchine Moderne: Architettura di von Neumann, Transistor, Circuiti Integrati, CPU, RAM, Bus di Controllo/Dati/Indirizzi.

Indirizzamento RAM: accesso diretto, relazione tra capacità della RAM e larghezza (in bit) del bus indirizzi.

Ciclo della CPU: Fetch-Decode-Execute, Datapath, Frequenza di Clock, Ciclo di clock, registri.

Attività pratiche: Su windows, installare CPU-Z, aprirlo e salvare il report del proprio computer in html. Su mac: <https://support.apple.com/it-it/guide/mac-help/syspr35536/mac>

Video (Mercoledì 12)

Vedere i video:

<https://www.youtube.com/watch?v=H4G4WLssTWg&t=28s>

<https://www.youtube.com/watch?v=vgPFzb1Bh7w> (Intel lez.1)!

https://www.youtube.com/watch?v=o_WXTRS2qTY (Intel lez.2)

<https://www.youtube.com/watch?v=d86ws7mQYIg> (PC)

<https://www.youtube.com/watch?v=7J7X7aZvMXQ> (DRAM)

<https://www.youtube.com/watch?v=dX9CGRZwD-w> (Chip)³

I video di *Branch Education* sono molto dettagliati (più di quello richiesto per il corso) ma bellissimi.

³mentre guardate il video segnatevi ogni termine che NON capite; per i video in inglese, potete usare i sottotitoli in Italiano generati da IA.

Week 3

Hardware

Oltre a CPU e RAM, serve una memoria **permanente** che consenta la memorizzazione delle informazioni a lungo termine.

Tecnologie: Dischi Magnetici (Hard-Drive) capienti ma lenti, Solid State Drive (SSD) (memorie flash) molto più veloci ma ancora con prezzo superiore a quello degli HD.

Verificare su Amazon, controllando su vari modelli, il prezzo medio per TB di un harddisk ad esempio da 8TB e/o il prezzo per TB di vari SSD.

Perifere: Video (HDMI), Standard di connessione interno PCI-Express https://en.wikipedia.org/wiki/PCI_Express) e bus esterno USB <https://it.wikipedia.org/wiki/USB>. Vedere tabelle.

Rappresentazione Istruzioni (Lunedì 17)

CPU come *interprete* di comandi in linguaggio macchina

Codifica delle istruzioni in binario, linguaggio Assembly, ISA
(Instruction Set Architecture)

Esempio: Emulatore Processore 8086:

<https://yjdock2.github.io/8086-emulator-web/compile>

Programmi in Assembly: livello più basso di programmazione, le istruzioni dipendono dal modello di processore -> se si cambia processore spesso si deve riscrivere il codice.

Per questo i programmi moderni sono scritti in linguaggi di *alto livello*, che non dipendono dall'hardware. Non sono però direttamente eseguibili e vanno tradotti in linguaggio macchina.

Latenze

Operazione	Tempo (unità)
1 CPU cycle	0.3 ns
Level 1 cache access	0.9 ns
Level 2 cache access	2.8 ns
Level 3 cache access	12.9 ns
Main memory access	120 ns
Solid-state disk I/O	50-150 μ s
Rotational disk I/O	1-10 ms
Internet: SF to NYC	40 ms
Internet: SF to UK	81 ms
Internet: SF to Australia	183 ms
OS virtualization reboot	4 s
SCSI command time-out	30 s
Hardware virtualization reboot	40 s
Physical system reboot	5 min

Tabella 1: Tempi di accesso e durata delle operazioni

Latenze

Operazione	Tempo (unità)	Tempo in anni
1 CPU cycle	0.3 ns	1 s
Level 1 cache access	0.9 ns	3 s
Level 2 cache access	2.8 ns	9 s
Level 3 cache access	12.9 ns	43 s
Main memory access	120 ns	6 min
Solid-state disk I/O	50-150 μ s	2-6 giorni
Rotational disk I/O	1-10 ms	1-12 mesi
Internet: SF to NYC	40 ms	4 anni
Internet: SF to UK	81 ms	8 anni
Internet: SF to Australia	183 ms	19 anni
OS virtualization reboot	4 s	423 anni
SCSI command time-out	30 s	3000 anni
Hardware virtualization reboot	40 s	4000 anni
Physical system reboot	5 min	32 millenni

Tabella 1: Tempi di accesso e durata delle operazioni

Sistemi Operativi

Il sistema operativo è il software di base che fornisce servizi per il livello applicativo, gestisce in modo completo l'hardware, creando una **macchina virtuale** più semplice da programmare e da usare.

Sistemi Operativi:

https://it.wikipedia.org/wiki/Sistema_operativo

Ogni sistema operativo ha vari livelli, uno più interno chiamato **Kernel** implementa le funzionalità più a contatto con l'hardware e le risorse del computer, e consente (tramite istruzioni privilegiate) di gestire la *multi-programmazione*.

Il livello più esterno è la GUI (Graphics User Interface), l'insieme di funzionalità grafiche che consentono un suo del computer ad utenti non esperti.

Sistemi Operativi

Un livello importante del sistema operativo è il *File System*, l'insieme di astrazioni fornite dal sistema per lavorare con *file* e *cartelle* indipendentemente dal dispositivo su cui sono memorizzati.

Un **file** è una sequenza di byte che rappresenta delle informazioni tra loro correlate, possono essere dati (testo, immagini, audio, video, etc etc) che programmi.

Un programma è costituito dall'insieme dei file (binari, librerie, file di configurazione) necessari al suo funzionamento, quando viene posto in esecuzione esso diventa un *processo*, occupa parte della RAM e parte del tempo del processore.

Nei sistemi multiprogrammati e multi-tasking a condivisione di tempo, il sistema operativo gestisce in modo ottimale tutti i core della CPU e lo spazio della RAM (e dei livelli di cache) per massimizzare le funzionalità del sistema.

Sistema Operativo

I sistemi operativi forniscono anche funzionalità importanti per la programmazione di applicazioni, tramite *librerie* di codice e *chiamate di sistema*.

Queste funzionalità consentono l'accesso e l'uso delle risorse della macchina *mediate* dal sistema operativo, a più programmi, in modo più facile che accedervi direttamente.

Ogni sistema operativo oltre all'interfaccia grafica offre un **Terminale di Comandi** che consente di eseguire spesso più attività complesse tramite sequenze di comandi.

Sistemi Operativi di largo utilizzo: Windows (ver. 10,11,Server), MacOS (IpadOS, iOS), Linux (Ubuntu, Fedora, Debian), Android (Kernel linux), etc.

Terminale (MacOS)

Per iniziare:

<https://ftp-mac.com/it/mac-terminal-commands.html>

Installate HomeBrew da `/bin/bash-c"$(curl-fsSLhttps://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"`, fate attenzione ad eseguire i comandi che vengono proposti alla fine.

Installate Oh-My-Zsh: <https://ohmyz.sh> Installate con HomeBrew: bat e eza

Terminale (Windows)

Installate dallo store "Windows Terminal" <https://apps.microsoft.com/detail/9N0DX20HK701?hl=it-it&gl=IT&ocid=pdpshare> (già installato in Windows 11).

Aprire il Terminale e installare **winget** <https://learn.microsoft.com/it-it/windows/package-manager/winget/> digitando:

```
Add-AppxPackage -RegisterByFamilyName -MainPackage  
Microsoft.DesktopAppInstaller_8wekyb3d8bbwe.
```

 seguito dal tasto Invio. Chiudere e riaprire il terminale e verificare se il comando `> winget` funziona. Leggere nella pagina del comando le informazioni sul suo utilizzo.

Andate nel menu Impostazioni del Terminale (CTRL+), alla voce Avvio->Profilo predefinito selezionate 'Prompt dei comandi'.

Installate: `chrisant996.Clink`, `JanDeDobbeleer.OhMyPosh`,
`eza-community.eza`, `sharkdp.bat`, `Python.Python.3.13`,
`Microsoft.VisualStudioCode`

Week 4 (24,26 Marzo)

Prompt dei Comandi, video:

<https://www.youtube.com/watch?v=Khpdwk4ksvQ>

facoltativo **Oh-My-Posh**:

<https://www.youtube.com/watch?v=2hnyshf7G6o>, **NerdFont**:

<https://www.youtube.com/watch?v=JNMvRv69pGE>

Pacchetto Office tramite Unisalento:

<https://www.unisalento.it/-/microsoft-office-365>

Riprodurre: <https://massimilianomoraca.me/blog/pygis-blog/mappatura-dellandamento-del-pm10-nel-nord-italia/>

Week 5 (31/3, 2/4)

Python uso di ipython, jupyter e jupyter-lab, uso di google colab e jupyter dentro vscode,

tipi interi, float, complex, booleani operazioni su questi. uso del modulo math

costrutti di controllo del flusso: if, if..else, if..elif..else, while. terminazione o cicli infiniti.

algoritmi: massimo comune divisore

definizione di nuove funzioni, chiamata di funzione, funzioni con input: parametri formali vs parametri attuali. funzioni che restituiscono valori.

scrivere il mcd come funzione.

scrivere una funzione che calcola il fattoriale di un intero n.

altri: <https://www.programmareinpython.it/esercizi-python/>

iniziare le liste per mercoledì

Codice delle lezioni disponibile nei fogli jupyter nella cartella Python a sinistra.

Prossima lezione: liste, numpy e matplotlib. (vedere su internet).

Numpy Tutorial: <https://jalammar.github.io/visual-numpy/>,
https://numpy.org/doc/2.2/user/absolute_beginners.html

Matplotlib: <https://github.com/rougier/scientific-visualization-book/tree/master/pdf>

Seaborn: <https://seaborn.pydata.org/>

Esempi

I notebook di jupyter sono estremamente popolari per creare documenti come questo:

https://tskit.dev/tutorials/what_is.html#sec-what-is

aprite le sezioni indicate come code, ovviamente non serve capire il codice ma, capite l'utilità e efficacia di documenti che contengono testo, grafici e codice insieme.

Appetito [https:](https://ncbi-taxonomist.readthedocs.io/en/latest/basics.html)

[/ncbi-taxonomist.readthedocs.io/en/latest/basics.html](https://ncbi-taxonomist.readthedocs.io/en/latest/basics.html)

<https://etetoolkit.org/docs/2.3/index.html>

<https://geo-python-site.readthedocs.io/en/latest/>

https://youtu.be/G6N08AVMuuU?si=mGKlqGYgz_vhGzu1

Riferimenti

Computer Science: https://wiki2.org/en/Computer_science

Binary Code: https://wiki2.org/en/Binary_code

History: https://wiki2.org/en/Binary_number#History

Storia: https://it.wikipedia.org/wiki/Sistema_di_numerazione

Informatica Storia:

https://it.wikipedia.org/wiki/Storia_dell%27informatica

Storia del Computer:

https://it.wikipedia.org/wiki/Storia_del_computer

Architettura del Calcolatore:

https://wiki2.org/en/Computer_architecture

Alan Turing: https://it.wikipedia.org/wiki/Alan_Turing

John Von Neumann:

https://it.wikipedia.org/wiki/John_von_Neumann

Week 6 (7/4, 9/4)

Week 7 (12/4)

Introduzione a Excel

Introduzione ad Excel.

- Foglio Elettronico, Fogli, Celle, Riferimenti.
- Contenuti di una cella, formattazione colonne, righe.
- Contenuti numerici e testuali.
- Formule, Riferimenti Relativi e Assoluti.
- Formule: SOMMA, MIN, MAX, MEDIA
- Esercizio sui Riferimenti: Tabelline.

Week 8 (7/5)

Casi D'uso: Sistema Elettrico Nazionale

Comprendere il Sistema Elettrico Nazionale, nelle sue componenti di generazione, trasporto, consumo.

Comprendere la differenza tra Energia (in Wh) e Potenza (in W).

<https://dati.terna.it>

Week 9 (12-14/5)

Week 10 (19-21/5)

Week 11 (26-28/5)

Week 12 (3-4/6)

Accenni alla Computabilità

Cosa significa *calcolare*? cosa è possibile calcolare in modo *automatico* (con una macchina)?

Queste sono due domande fondamentali a cui gli informatici hanno dato risposta prima ancora di riuscire a costruire i moderni computer, il *papà* dell'Informatica è Alan Turing, matematico e logico Inglese, rappresentato nel film "The Imitation Game".

Turing, in un articolo del 1936 presenta un modello Teorico di *macchina di calcolo* (da quel momento detta **Macchina di Turing**), è un modello astratto, semplicissimo, ma capace di catturare l'essenza di ogni modello di computazione automatica.

Macchina di Turing

Una macchina di Turing è un semplice automa a stati finiti, con una memoria limitata (finita) ed un nastro infinito diviso in celle su cui è possibile scrivere caratteri da un alfabeto finito di simboli. La macchina può spostarsi di una casella sulla destra/sinistra, leggere un simbolo dal nastro, ed eventualmente riscriverlo.

Questo modello così semplice corrisponde alle prime macchine di calcolo con *programma cablato*, la macchina infatti può svolgere solo una computazione pre-assegnata. Ad ogni macchina quindi corrisponde una specifica funzione computabile.

Macchina di Turing

Turing dimostra l'esistenza di una speciale macchina, detta *Universale* capace di leggere la descrizione di una qualunque altra macchina, posta sul nastro di input, insieme ad i dati su cui lavorare, il cui comportamento simula (o interpreta) proprio il comportamento della macchina posta sul nastro.

Questo concetto, descrive tutti i calcolatori moderni, a programma memorizzato. Una CPU non è altro che una macchina universale, non più potente quindi di una macchina di Turing dal punto di visto qualitativo anche se enormemente più veloce.

Sono stati presentati altri modelli di calcolo, ma si sono dimostrati essere tutti equivalenti al modello basato sulle macchine di Turing. Da cui:

Tesi di Church-Turing: Tutto ciò che è calcolabile, è calcolabile da una macchina di Turing.

Risoluzione in laboratorio di domande poste negli appelli precedenti.

- Conversione numerica binario $\leftrightarrow$ decimale, somme e sottrazioni in binario.
- Domande sull'architettura dei calcolatori.
- Domande sulle reti di calcolatori
- Domande su compiti da svolgere in Excel.
- Domande su esercizi da scrivere in Python.

Week 13 (9-11/6)

Appendix

<https://docs.kepler.gl/docs/keplergl-jupyter>

<https://earth-env-data-science.github.io/intro.html>

[https://courses.spatialthoughts.com/python-dataviz.html#
creating-charts](https://courses.spatialthoughts.com/python-dataviz.html#creating-charts)